LAB 5

Static Code Analysis

Pharmacy Manager — Python

1. Objective

The objective of this lab is to improve the quality, readability, maintainability, and security of a Python program by applying static analysis. Pylint was used to identify code-quality and design concerns, Flake8 to check PEP 8 formatting, and Bandit to detect potentially unsafe coding patterns.

2. Program Overview

The Pharmacy Manager program maintains medicine stock and demonstrates medicine dispensing, quantity lookup, expiry checks, bill calculation, dosage calculation, patient ID generation, prescription number generation, stock persistence, dispensing labels, pharmacist login, and stock reporting.

3. Tools and Commands

Tool / test	Command
Pylint	python -m pylint pharmacy_manager.py
Bandit	python -m bandit pharmacy_manager.py
Flake8	python -m flake8 pharmacy_manager.py
Program test	python pharmacy_manager.py

4. Before-and-After Results

The initial reports showed multiple style, maintainability, and security warnings. After the code was revised, the final terminal screenshots show the following results:

Analysis tool	Before	After
Pylint	4.83/10	10.00/10
Flake8	Multiple F401, E302, E501, E722, E305 findings	No output / no reported violations
Bandit	Multiple findings, including eval, MD5, random, pickle and shell execution	No issues identified; 0 findings
Runtime test	Program needed cleanup and safer implementations	Program runs and prints stock/report output

Note: The initial Bandit report displayed in the screenshots scanned 89 lines and reported multiple findings. The final report scanned 182 lines and reported zero issues. The final Pylint score is 10.00/10, exceeding the lab requirement of more than 9.5.

5. Issue Identification and Fix Table

No.	Issue / code	Type / severity	Finding	Fix applied
1	B307 — use of eval()	Security / High	Dosage formula was evaluated with eval(), which can execute arbitrary Python expressions.	Replaced eval-based calculation with a restricted, validated arithmetic approach so the input is not

				executed as arbitrary code.
2	B324 — MD5 hash	Security / High	MD5 was used to create a patient identifier even though it is unsuitable for security-sensitive hashing.	Changed patient identifier generation to SHA-256.
3	B311 — random for prescription number	Security / Low	random.randint() was used to generate a prescription number.	Used the secrets module to generate a less predictable prescription identifier.
4	B301/B403 — pickle deserialization/import	Security / Medium/Low	Pickle can execute unsafe deserialization behavior when data is untrusted.	Replaced pickle-based stock persistence with JSON serialization and loading.
5	B605 — shell command in label printing	Security / High	os.system() invoked a shell command to print the dispensing label.	Replaced shell-based printing with normal Python output, avoiding shell invocation.
6	B105 — hard-coded password	Security / Low	The pharmacist password was embedded directly in the source code.	Moved password configuration to the DB_PASSWORD environment variable; set it in the terminal when testing login.
7	F401 — unused imports	Style / Maintainability	Unused imports such as sys and logging generated Flake8 warnings.	Removed imports that were not needed.
8	E302/E305 — blank-line spacing	Style	Function definitions and class/function boundaries did not follow PEP 8 spacing rules.	Corrected blank lines around definitions.
9	E501 / C0301 — long lines	Style	Some lines exceeded the configured line-length limit.	Wrapped long statements and formatted them across multiple lines.
10	E722 / W0702 — bare except	Reliability / Maintainability	A bare except clause caught all exceptions without specifying the expected failure.	Narrowed exception handling to appropriate exception types and preserved useful error behavior.
11	C0114/C0116 — missing docstrings	Maintainability	The module and functions lacked documentation strings.	Added module/function documentation where required by the final Pylint configuration.
12	C0103 — naming convention	Maintainability	Several function names used camelCase rather than Python snake_case.	Renamed functions to snake_case and updated their references.
13	W0102 — mutable default argument	Correctness / Maintainability	A list was used as a default argument, which can be shared across function calls.	Changed the default to None and created a fresh list inside the function.

6. Final Program Verification

The final program output confirms that the application executes and produces expected demonstration output:

- An attempt to dispense an unregistered medicine (Insulin) is handled and reports a failure message.
- Paracetamol stock is shown as 170 units after the demo dispensing operation.
- The dosage expression “250 * 2” returns 500.0 mg/day.
- A SHA-256 patient identifier and an RX prescription number are generated.
- A dispensing label and pharmacy stock report are printed.

The login result in the screenshot is False because DB_PASSWORD was not set in that terminal session. This is configuration-dependent and does not mean the static-analysis checks failed. For a separate login test in Windows CMD, run: set DB_PASSWORD=pharma@123, then run python pharmacy_manager.py. Do not store real credentials in source code.

7. Reflection

7.1 Why is static code analysis useful?

Static analysis finds potential defects without requiring every code path to be executed. It helps detect risky APIs, poor exception handling, style violations, and maintainability problems early, making code easier to review and safer to change.

7.2 What did each tool contribute?

Pylint highlighted naming, documentation, design, and code-quality issues. Flake8 identified formatting violations such as unused imports, long lines, and incorrect blank-line spacing. Bandit identified security-sensitive patterns such as eval, weak hashing, predictable random values, pickle deserialization, and shell execution.

7.3 What was the most important fix?

The most important fixes were removing arbitrary expression evaluation and shell execution, replacing unsafe serialization, and using stronger mechanisms for identifiers. These changes reduce the risk of untrusted input affecting program execution and improve the security of the application.

7.4 What did you learn?

I learned that a program can run successfully and still contain security or maintainability problems. Running several complementary tools gives a more complete picture than relying on runtime testing alone. I also learned to rerun the tools after changes and use their reports as evidence that the issues were addressed.

8. Git Commit Evidence

The lab handout requires three commits covering changes prompted by the three analysis tools. The screenshots supplied do not show Git commit history, so the commits should be created and verified separately before submission. Suggested commit messages:

- git add pharmacy_manager.py && git commit -m "Fix security findings reported by Bandit"
- git add pharmacy_manager.py && git commit -m "Resolve Pylint code quality warnings"
- git add pharmacy_manager.py && git commit -m "Fix Flake8 formatting violations"

If all code changes have already been included in one commit, do not create misleading commits that claim separate historical changes. Follow the lab's required commit sequence using actual staged changes, or ask the instructor how to document the work accurately.

9. Screenshots

The following terminal screenshots are included as evidence. They show the initial findings, the corrected reports, and the final program execution.

```
C:\Users\jaina\Downloads\L5_55_pharmacy-manager-main\L5_55_pharmacy-manager-main>python -m pylint pharmacy_manager.py
************* Module pharmacy_manager
pharmacy_manager.py:87:0: C0301: Line too long (111/100) (line-too-long)
pharmacy_manager.py:1:0: C0114: Missing module docstring (missing-module-docstring)
pharmacy_manager.py:13:0: C0116: Missing function or method docstring (missing-function-docstring)
pharmacy_manager.py:13:0: C0103: Function name "addMedicine" doesn't conform to snake_case naming style (invalid-name)
pharmacy_manager.py:13:0: W0102: Dangerous default value [] as argument (dangerous-default-value)
pharmacy_manager.py:19:17: C0209: Formatting a regular string which could be an f-string (consider-using-f-string)
pharmacy_manager.py:13:0: R1710: Either all return statements in a function should return an expression, or none of them should. (inconsistent-return-statements)
pharmacy_manager.py:22:0: C0116: Missing function or method docstring (missing-function-docstring)
pharmacy_manager.py:22:0: C0103: Function name "dispenseMedicine" doesn't conform to snake_case naming style (invalid-name)
pharmacy_manager.py:27:4: W0702: No exception type(s) specified (bare-except)
pharmacy_manager.py:30:0: C0116: Missing function or method docstring (missing-function-docstring)
pharmacy_manager.py:30:0: C0103: Function name "getQuantity" doesn't conform to snake_case naming style (invalid-name)
pharmacy_manager.py:33:0: C0116: Missing function or method docstring (missing-function-docstring)
pharmacy_manager.py:33:0: C0103: Function name "isExpired" doesn't conform to snake_case naming style (invalid-name)
pharmacy_manager.py:35:4: R1703: The if statement can be replaced with 'return bool(test)' (simplifiable-if-statement)
pharmacy_manager.py:35:4: R1705: Unnecessary "else" after "return", remove the "else" and de-indent the code inside it (no-else-return)
pharmacy_manager.py:40:0: C0116: Missing function or method docstring (missing-function-docstring)
pharmacy_manager.py:40:0: C0103: Function name "billAmount" doesn't conform to snake_case naming style (invalid-name)
pharmacy_manager.py:46:0: C0116: Missing function or method docstring (missing-function-docstring)
pharmacy_manager.py:46:0: C0103: Function name "calculateDosage" doesn't conform to snake_case naming style (invalid-name)
pharmacy_manager.py:48:11: W0123: Use of eval (eval-used)
pharmacy_manager.py:50:0: C0116: Missing function or method docstring (missing-function-docstring)
pharmacy_manager.py:50:0: C0103: Function name "patientId" doesn't conform to snake_case naming style (invalid-name)
pharmacy_manager.py:53:0: C0116: Missing function or method docstring (missing-function-docstring)
pharmacy_manager.py:53:0: C0103: Function name "generateRxNumber" doesn't conform to snake_case naming style (invalid-name)
pharmacy_manager.py:56:0: C0116: Missing function or method docstring (missing-function-docstring)
pharmacy_manager.py:56:0: C0103: Function name "saveStock" doesn't conform to snake_case naming style (invalid-name)
pharmacy_manager.py:57:8: R1732: Consider using 'with' for resource-allocating operations (consider-using-with)
pharmacy_manager.py:61:0: C0116: Missing function or method docstring (missing-function-docstring)
pharmacy_manager.py:61:0: C0103: Function name "loadStock" doesn't conform to snake_case naming style (invalid-name)
pharmacy_manager.py:63:4: W0603: Using the global statement (global-statement)
pharmacy_manager.py:62:8: R1732: Consider using 'with' for resource-allocating operations (consider-using-with)
pharmacy_manager.py:67:0: C0116: Missing function or method docstring (missing-function-docstring)
pharmacy_manager.py:67:0: C0103: Function name "printLabel" doesn't conform to snake_case naming style (invalid-name)
pharmacy_manager.py:70:0: C0116: Missing function or method docstring (missing-function-docstring)
pharmacy_manager.py:70:0: C0103: Function name "expiringSoon" doesn't conform to snake_case naming style (invalid-name)
pharmacy_manager.py:78:0: C0116: Missing function or method docstring (missing-function-docstring)
pharmacy_manager.py:78:0: C0103: Function name "pharmacistLogin" doesn't conform to snake_case naming style (invalid-name)
pharmacy_manager.py:79:4: R1703: The if statement can be replaced with 'return bool(test)' (simplifiable-if-statement)
pharmacy_manager.py:79:4: R1705: Unnecessary "else" after "return", remove the "else" and de-indent the code inside it (no-else-return)
pharmacy_manager.py:84:0: C0116: Missing function or method docstring (missing-function-docstring)
pharmacy_manager.py:84:0: C0103: Function name "printStock" doesn't conform to snake_case naming style (invalid-name)
pharmacy_manager.py:89:0: C0116: Missing function or method docstring (missing-function-docstring)
pharmacy_manager.py:2:0: W0611: Unused import sys (unused-import)
pharmacy_manager.py:6:0: W0611: Unused import logging (unused-import)

-----------------------------------
Your code has been rated at 4.83/10
```

Figure 1. Initial Bandit report showing security findings in the original code.

```
C:\Users\jaina\Downloads\L5_55_pharmacy-manager-main\L5_55_pharmacy-manager-main>python -m flake8 pharmacy_manager.py
pharmacy_manager.py:2:1: F401 'sys' imported but unused
pharmacy_manager.py:6:1: F401 'logging' imported but unused
pharmacy_manager.py:13:1: E302 expected 2 blank lines, found 1
pharmacy_manager.py:19:80: E501 line too long (81 > 79 characters)
pharmacy_manager.py:22:1: E302 expected 2 blank lines, found 1
pharmacy_manager.py:27:5: E722 do not use bare 'except'
pharmacy_manager.py:30:1: E302 expected 2 blank lines, found 1
pharmacy_manager.py:33:1: E302 expected 2 blank lines, found 1
pharmacy_manager.py:40:1: E302 expected 2 blank lines, found 1
pharmacy_manager.py:46:1: E302 expected 2 blank lines, found 1
pharmacy_manager.py:50:1: E302 expected 2 blank lines, found 1
pharmacy_manager.py:53:1: E302 expected 2 blank lines, found 1
pharmacy_manager.py:56:1: E302 expected 2 blank lines, found 1
pharmacy_manager.py:61:1: E302 expected 2 blank lines, found 1
pharmacy_manager.py:67:1: E302 expected 2 blank lines, found 1
pharmacy_manager.py:70:1: E302 expected 2 blank lines, found 1
pharmacy_manager.py:78:1: E302 expected 2 blank lines, found 1
pharmacy_manager.py:84:1: E302 expected 2 blank lines, found 1
pharmacy_manager.py:87:80: E501 line too long (111 > 79 characters)
pharmacy_manager.py:89:1: E302 expected 2 blank lines, found 1
pharmacy_manager.py:98:80: E501 line too long (85 > 79 characters)
pharmacy_manager.py:108:1: E305 expected 2 blank lines after class or function definition, found 1
```

Figure 2. Initial Flake8 report showing formatting and style violations.

```
C:\Users\jaina\Downloads\L5_55_pharmacy-manager-main\L5_55_pharmacy-manager-main>python -m bandit pharmacy_manager.py
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.13.7
Run started:2026-10-14 06:18:30.486523+00:00

Test results:
>> Issue: [B403:blacklist] Consider possible security implications associated with pickle module.
   Severity: Low   Confidence: High
   CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_imports.html#b403-import-pickle
   Location: .\pharmacy_manager.py:3:0
2       import sys
3       import pickle
4       import hashlib

--------------------------------------------------
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'pharma@123'
   Severity: Low   Confidence: Medium
   CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b105_hardcoded_password_string.html
   Location: .\pharmacy_manager.py:11:14
10      medicines = {}
11      DB_PASSWORD = "pharma@123"
12

--------------------------------------------------
>> Issue: [B110:try_except_pass] Try, Except, Pass detected.
   Severity: Low   Confidence: High
   CWE: CWE-703 (https://cwe.mitre.org/data/definitions/703.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b110_try_except_pass.html
   Location: .\pharmacy_manager.py:27:4
26                  del medicines[name]
27          except:
28              pass
29


--------------------------------------------------
```

Figure 3. Initial Pylint report: 4.83/10 with code-quality warnings.

```
>> Issue: [B307:blacklist] Use of possibly insecure function - consider using safer ast.literal_eval.
   Severity: Medium   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b307-eval
   Location: .\pharmacy_manager.py:48:11
47          # formula comes from the doctor's note, e.g. "250 * 2"
48          return eval(formula)
49

--------------------------------------------------
>> Issue: [B324:hashlib] Use of weak MD5 hash for security. Consider usedforsecurity=False
   Severity: High   Confidence: High
   CWE: CWE-327 (https://cwe.mitre.org/data/definitions/327.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b324_hashlib.html
   Location: .\pharmacy_manager.py:51:11
50      def patientId(name, phone):
51          return hashlib.md5((name + phone).encode()).hexdigest()
52

--------------------------------------------------
>> Issue: [B311:blacklist] Standard pseudo-random generators are not suitable for security/cryptographic purposes.
   Severity: Low   Confidence: High
   CWE: CWE-330 (https://cwe.mitre.org/data/definitions/330.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b311-random
   Location: .\pharmacy_manager.py:54:23
53      def generateRxNumber():
54          return "RX-" + str(random.randint(100000, 999999))
55

--------------------------------------------------
>> Issue: [B301:blacklist] Pickle and modules that wrap it can be unsafe when used to deserialize untrusted data, possible security issue.
   Severity: Medium   Confidence: High
   CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b301-pickle
   Location: .\pharmacy_manager.py:64:16
63          global medicines
64          medicines = pickle.load(f)
65          f.close()

--------------------------------------------------
>> Issue: [B605:start_process_with_a_shell] Starting a process with a shell, possible injection detected, security issue.
   Severity: High   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b605_start_process_with_a_shell.html
   Location: .\pharmacy_manager.py:68:4
67      def printLabel(name):
68          os.system("echo Dispensing label: " + name)
69

--------------------------------------------------
```

Figure 4. Final execution and Bandit verification: no security issues identified.

```
--------------------------------------------------
>> Issue: [B605:start_process_with_a_shell] Starting a process with a shell, possible injection detected, security issue.
   Severity: High   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b605_start_process_with_a_shell.html
   Location: .\pharmacy_manager.py:68:4
67      def printLabel(name):
68          os.system("echo Dispensing label: " + name)
69

--------------------------------------------------

Code scanned:
        Total lines of code: 89
        Total lines skipped (#nosec): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 4
                Medium: 2
                High: 2
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 1
                High: 7
Files skipped (0):

C:\Users\jaina\Downloads\L5_55_pharmacy-manager-main\L5_55_pharmacy-manager-main>copy pharmacy_manager.py pharmacy_manager_backup.py
        1 file(s) copied.
```

Figure 5. Final Flake8 and Pylint verification: no Flake8 output and Pylint 10.00/10.

```
C:\Users\jaina\Downloads\L5_55_pharmacy-manager-main\L5_55_pharmacy-manager-main>python -m bandit pharmacy_manager.py
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.13.7
Run started:2026-10-14 07:06:26.603547+00:00

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 182
        Total lines skipped (#nosec): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
Files skipped (0):

C:\Users\jaina\Downloads\L5_55_pharmacy-manager-main\L5_55_pharmacy-manager-main>python pharmacy_manager.pynotepad pharmacy_manager.py
python: can't open file 'C:\\Users\\jaina\\Downloads\\L5_55_pharmacy-manager-main\\L5_55_pharmacy-manager-main\\pharmacy_manager.pynotepad': [Errno 2] No such file or directory

C:\Users\jaina\Downloads\L5_55_pharmacy-manager-main\L5_55_pharmacy-manager-main>notepad pharmacy_manager.py

C:\Users\jaina\Downloads\L5_55_pharmacy-manager-main\L5_55_pharmacy-manager-main>python -m flake8 pharmacy_manager.py

C:\Users\jaina\Downloads\L5_55_pharmacy-manager-main\L5_55_pharmacy-manager-main>python -m pylint pharmacy_manager.py

--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

Figure 6. Additional initial Bandit findings, including eval, MD5, random, pickle, and shell use.

```
C:\Users\jaina\Downloads\L5_55_pharmacy-manager-main\L5_55_pharmacy-manager-main>python -m bandit pharmacy_manager.py
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.13.7
Run started:2026-10-14 07:07:45.478643+00:00

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 182
        Total lines skipped (#nosec): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
Files skipped (0):

C:\Users\jaina\Downloads\L5_55_pharmacy-manager-main\L5_55_pharmacy-manager-main>python pharmacy_manager.py
Dispensing failed: 'Medicine not found: Insulin'
Paracetamol stock: 170
Amoxicillin expired? False
Bill: 45.0
Dosage (mg/day): 500.0
Patient ID: 5cad00777782834f6d50fe7189fc142c3ce08a44b37df712dc07115ab964788c
Rx number: RX-462902
Login: False
Dispensing label: Paracetamol 500mg
===== Pharmacy Stock Report =====
Paracetamol 500mg -> 170 units, Rs. 1.5 exp 2027-06-30
Amoxicillin 250mg -> 40 units, Rs. 6.0 exp 2027-01-15
Cetirizine 10mg -> 10 units, Rs. 2.0 exp 2027-03-31
Vitamin C 500mg -> 25 units, Rs. 0.0 exp 2099-12-31

C:\Users\jaina\Downloads\L5_55_pharmacy-manager-main\L5_55_pharmacy-manager-main>^A
```

Figure 7. Final program output confirming the pharmacy manager runs.